COSC 4P96 Group Project: PSO-Optimized Pseudo-Labeling for Semi-Supervised Image Classification

Vijay Panchal (7225949), Manu Saini (7092158)

***Abstract*—Semi-supervised learning via pseudo-labeling [1] can substantially reduce the need for labeled data, yet its effectiveness is sensitive to hyperparameters such as the confidence threshold, pseudo-label budget, and pseudo-label weight. This paper investigates whether Particle Swarm Optimization (PSO) [2] can automatically discover hyperparameter configurations that outperform carefully designed manual strategies for a weighted pseudo-labeling pipeline on CIFAR-10. We train a lightweight CNN under six labeled-data fractions (5%–50%), comparing six expert-crafted manual strategies against a PSO-based search over a four-dimensional hyperparameter space, each evaluated across five random seeds. Our results show that well-chosen manual strategies achieve competitive or marginally higher test accuracy than PSO across all labeling regimes, with the gap narrowing as labeled data increases. However, manual tuning requires domain expertise to design effective strategies, whereas PSO reaches comparable performance without prior knowledge at roughly twice the computational cost. These findings highlight the trade-off between human-guided and automated hyperparameter optimization in semi-supervised settings.**

I. Introduction

A. Semi-Supervised Learning and Pseudo-Labeling

Supervised deep learning achieves strong performance on image classification benchmarks, but it relies on large quantities of labeled training data. Obtaining high-quality labels is often expensive and time-consuming, requiring domain experts or crowd-sourcing pipelines [3]. Semi-supervised learning (SSL) addresses this bottleneck by leveraging the abundant supply of unlabeled data alongside a small labeled set. Among SSL methods, pseudo-labeling [1] is one of the most widely adopted due to its simplicity and compatibility with standard training pipelines. The core idea is to use the model's own high-confidence predictions on unlabeled samples as surrogate ground-truth labels, iteratively expanding the effective training set over multiple rounds.

Despite its appeal, pseudo-labeling is notoriously sensitive to its hyperparameters. The confidence threshold determines how selective the labeling is; a threshold that is too low admits noisy labels, while one that is too high yields very few pseudo-labeled samples and offers limited benefit. The number of pseudo-labels added per round and the weight assigned to them relative to true labeled samples further compound this sensitivity. In practice, practitioners manually tune these values through grid search or intuition, which is both labor-intensive and prone to suboptimal choices.

B. Motivation for Automated Hyperparameter Optimization

Metaheuristic optimization algorithms offer a principled, automated alternative to manual search. Particle Swarm Optimization (PSO) [2] is a population-based evolutionary algorithm inspired by the collective behavior of bird flocking. Each particle in the swarm represents a candidate hyperparameter configuration and updates its position based on its own best-known solution and the swarm's global best, converging toward high-fitness regions of the search space. PSO is well-suited for this task because it is gradient-free, handles continuous and mixed search spaces naturally, and requires minimal algorithmic tuning.

C. Contributions

This work makes the following contributions:

- We design and implement a weighted pseudo-labeling SSL pipeline on CIFAR-10 using a lightweight CNN, where pseudo-labeled samples are down- weighted relative to true labels to reduce the impact of labeling noise.
- We formulate hyperparameter tuning as a four-dimensional continuous optimization problem and implement PSO to search over confidence threshold, pseudo-label budget, pseudo-label weight, and learning rate.
- We compare PSO-optimized configurations against six manually designed strategies across six labeled-data fractions, evaluated with five random seeds each, providing a statistically grounded comparison.
- We analyze the cost-benefit trade-off between expert-guided and automated tuning in the low-data regime.

II. Related Work

A. Pseudo-Labeling

Pseudo-labeling was first proposed by Lee [1] as a straightforward self-training approach where the model assigns labels to unlabeled samples based on its most confident predictions, then treats those as real labels in the next round of training. What made it appealing was how little it changed the standard training setup — you just add more samples with predicted labels and keep going. It works well enough when the model starts from a reasonable place, but falls apart quickly if early predictions are noisy.

A number of follow-up methods have tried to address that weakness. Xie et al. [4] showed that adding noise during the student model's training (while keeping the teacher clean) led

to surprisingly strong results on ImageNet, essentially closing much of the gap to fully supervised training. Sohn et al. [5] went further with FixMatch, which only accepts a pseudo-label if the model is confident on a weakly augmented view, and then trains on a strongly augmented version of the same image. That combination of consistency regularization and aggressive augmentation pushed performance substantially higher with very few labels.

For our project we deliberately stuck with the simpler iterative formulation rather than adopting something like FixMatch. The reason is that we wanted the hyperparameters — threshold, budget, weight — to be the main variable under study, and adding augmentation or consistency losses would have introduced additional confounders that make it harder to isolate the effect of those choices.

B. Hyperparameter Optimization

Picking the right hyperparameters is one of those problems that sounds straightforward but gets painful fast, especially in SSL where performance can swing dramatically with small changes to the confidence threshold or pseudo-label budget. The classical approach is grid search, but that scales poorly as the number of parameters grows. Bayesian optimization [6] is a smarter alternative: it fits a surrogate model (usually a Gaussian process) to the results seen so far and uses that to decide where to sample next. It tends to find good configurations in fewer evaluations than grid or random search, but it assumes the objective is fairly stationary, which is not really the case here since the training set itself changes between rounds.

Random search [7] is worth mentioning because it consistently beats grid search in practice when only a few of the parameters actually matter — and it is trivially parallelizable. For our setting though, we suspected all four hyperparameters interact in non-trivial ways, which made a more directed search strategy like PSO a better fit.

C. PSO for Deep Learning

PSO has shown up in deep learning mostly for things like neural architecture search and learning rate scheduling [8]. The main appeal is that it does not need gradients, so it can optimize anything you can evaluate, and the swarm structure naturally explores multiple regions of the search space at once rather than following a single trajectory. The downside is that each fitness evaluation is a full training run, which gets expensive quickly.

In our work we apply PSO at the hyperparameter level: each particle encodes a configuration of threshold, budget, pseudo-label weight, and learning rate, and its fitness is the best validation accuracy from a short SSL run with that configuration. This is essentially the same black-box setup used in prior PSO-based HPO work [8], just applied to the less-explored context of semi-supervised learning.

III. Methodology

A. Dataset and Data Splits

We ran all our experiments on CIFAR-10 [9], which is a fairly standard choice for SSL research since it is small enough to train quickly but still non-trivial to classify. The dataset has 60,000 RGB images at 32×32 pixels split across 10 classes, with 50,000 for training and 10,000 held out for testing. We normalized each channel using the dataset's standard mean $(0.4914, 0.4822, 0.4465)$ and standard deviation $(0.2470, 0.2435, 0.2616)$, and deliberately left out any data augmentation. That might sound like a missed opportunity performance-wise, but we wanted the comparison between manual and PSO-tuned strategies to hinge purely on the hyperparameters, not on augmentation effects that would be hard to disentangle.

For the splits, we first carved out a stratified 10% of the training set as a fixed validation set — this is what both the SSL training loop and the PSO fitness function use for model selection. The remaining 90% was then divided into a labeled pool and an unlabeled pool depending on the labeled fraction ρ, which we varied across $\{5\%, 10\%, 15\%, 20\%, 30\%, 50\%\}$. Stratified splitting was used throughout so that each class stays proportionally represented regardless of how small ρ gets. Every split is seeded, and we run the whole thing over five different seeds to get a sense of how much results vary just due to randomness in data assignment.

B. Model Architecture

We kept the model deliberately simple. The goal was not to push state-of-the-art accuracy on CIFAR-10 but to have something fast enough to train dozens of times during a PSO run without the whole thing taking days. The network, which we called , has two convolutional blocks followed by a small fully connected head:

- **Block 1:** Conv2d(3→32, 3×3, pad=1) → ReLU → MaxPool(2×2)
- **Block 2:** Conv2d(32→64, 3×3, pad=1) → ReLU → MaxPool(2×2)
- **Classifier:** Flatten → Linear(4096→128) → ReLU → Linear(128→10)

After the two pooling layers the spatial dimensions go from 32×32 down to 8×8, giving a flattened feature vector of $64 \times 8 \times 8 = 4096$ dimensions before the classifier. It is not a large model by any measure, but for a project focused on the hyperparameter optimization side of things, it was the right call.

C. Weighted Pseudo-Labeling SSL Pipeline

The core idea of the pipeline is straightforward: train on whatever labeled data you have, use the model to label the most confident unlabeled samples, add those to the training set, and repeat. What we added on top of the basic recipe is a weighting scheme — pseudo-labeled samples are not treated the same as real labeled ones. Instead, they are down-weighted by a factor $\lambda_p \leq 1.0$ to reduce the damage if some of those pseudo-labels turn out to be wrong.

Concretely, the per-sample weighted cross-entropy loss at each training step is:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} w_i \cdot \ell_{\mathrm{CE}}(f(x_i; \theta),\, y_i) \tag{1}$$

where $w_i = 1.0$ for truly labeled samples and $w_i = \lambda_p$ for pseudo-labeled ones, $f(\cdot;\theta)$ is the CNN, and ℓ_{CE} is standard cross-entropy. The value of λ_p is one of the hyperparameters we optimize.

After training for E epochs in a given round, we run inference over the remaining unlabeled pool and collect all samples where the peak softmax probability exceeds τ. Those are ranked by confidence and the top K get added to the training set for the next round with their predicted label attached. Algorithm 1 lays this out step by step.

Algorithm 1 Weighted Pseudo-Labeling SSL

Require: Labeled set $\mathcal{L}$, unlabeled pool $\mathcal{U}$, threshold τ, budget K, weight λ_p, rounds R, epochs E, learning rate η

```
Initialize model f(·; θ)
P ← ∅ {pseudo-label accumulator}
for r = 1 to R do
    Train f on L ∪ P for E epochs using loss (1)
    C ← {(x, ŷ, p) : x ∈ U, p = max_c σ_c(f(x)) ≥ τ}
    Sort C by p descending; take top K
    P ← P ∪ C; U ← U \ C
end for
return Best model by validation accuracy
```

At the end of all rounds, we keep whichever model checkpoint achieved the highest validation accuracy, not necessarily the last one. Training uses the Adam optimizer [10], and we enabled automatic mixed precision on CUDA to speed things up where hardware allowed.

D. Hyperparameter Search Space

We identified four hyperparameters that we expected to most influence pseudo-labeling quality, and those are what both the manual strategies and PSO operate over:

- **Confidence threshold** $\tau \in [0.90, 0.99]$
- **Pseudo-label budget per round** $K \in [200, 1500]$
- **Pseudo-label weight** $\lambda_p \in [0.2, 1.0]$
- **Learning rate** $\eta \in [0.0003, 0.002]$

The bounds were chosen based on what seemed reasonable for a 10-class problem with a small model — for instance, thresholds below 0.90 tend to admit too much noise, while going above 0.99 at low labeling regimes often yields almost no pseudo-labels at all. During PSO evaluation we fixed the pipeline at $R = 2$ rounds and $E = 3$ epochs per round, which keeps each fitness evaluation fast enough to be practical. The manual strategies use the same budget so the comparison stays fair.

E. Particle Swarm Optimization

PSO works by maintaining a swarm of S candidate solutions (particles) that move around the search space guided by their own best-found position and the best position seen by anyone in the swarm. At each iteration, the velocity and position of particle i are updated as:

$$v_i^{t+1} = w \cdot v_i^t + c_1 r_1 (p_i^{\text{best}} - x_i^t) + c_2 r_2 (g^{\text{best}} - x_i^t) \quad (2)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (3)$$

Here $w = 0.7$ controls how much momentum carries over from the previous step, while $c_1 = c_2 = 1.5$ balance the pull toward the particle's own best versus the global best. The terms r_1 and r_2 are independent uniform random draws from $[0, 1]$ that inject some stochasticity into the updates. After each step, positions are clipped back to the search bounds if they drift outside.

We used a swarm of $S = 5$ particles over $T = 5$ iterations. Including the initialization phase, that works out to at most $S(T + 1) = 30$ full SSL runs per labeled fraction. The fitness score for each particle is simply the best validation accuracy recorded during its corresponding SSL run.

F. Manual Strategies

To give PSO a meaningful benchmark to beat, we designed six manual configurations by hand. They are meant to represent a range of intuitions a practitioner might have: some are conservative (high τ, small K, lower λ_p) to avoid noisy labels, some are more aggressive (lower τ, large K) to maximize the number of pseudo-labeled samples added per round, and others sit somewhere in the middle. The threshold spans from $\tau \leq 0.92$ on the aggressive end up to $\tau \geq 0.97$ for the conservative ones. Full parameter listings for all six strategies are in the folder of our repository.

G. Evaluation Protocol

Every experiment — both manual and PSO — is run with five different random seeds $\{42, 43, 44, 45, 46\}$, giving us enough repetitions to report a meaningful mean and standard deviation. For each run we evaluate the final model on the CIFAR-10 test set and record test accuracy and macro-averaged F1-score. Accuracy is our headline metric, but we included macro F1 as a sanity check to make sure neither method is quietly ignoring certain classes to inflate its overall number.

IV. Experimental Results

A. Manual vs. PSO-Optimized SSL

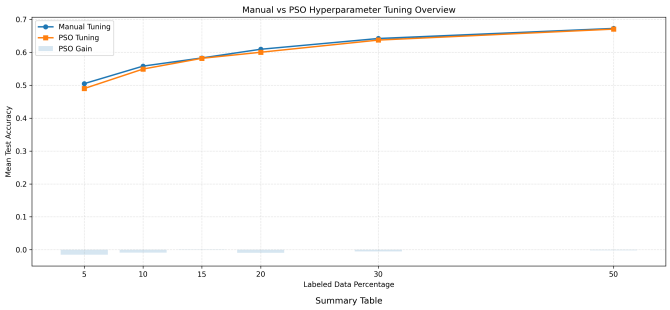
Table I shows the main comparison between the best manual strategy and PSO across all six labeled fractions. The first thing that stands out is how close the numbers are — in no regime does PSO beat the best manual configuration, but it never falls far behind either. At 5% labeled data the gap is about 1.5 percentage points, which is the largest difference we saw. By the time you get to 50% labeled data, the two are essentially tied at 67.25% vs. 67.04%.

It is also worth noting that PSO's standard deviation at 15% and 20% is noticeably higher than the manual strategy's. This suggests that with only 30 fitness evaluations the swarm has not fully converged — different seeds send it to different regions of the search space, and not all of them are equally good. The manual strategies, by contrast, are fixed configurations

TABLE I: Test Accuracy: Best Manual Strategy vs. PSO (mean $\pm$ std, 5 seeds)

Labeled Fraction	Best Manual Strategy	PSO Optimized
5%	0.5048 ± 0.0075	0.4899 ± 0.0065
10%	0.5580 ± 0.0033	0.5491 ± 0.0055
15%	0.5828 ± 0.0044	0.5817 ± 0.0111
20%	0.6092 ± 0.0066	0.6001 ± 0.0107
30%	0.6419 ± 0.0106	0.6371 ± 0.0067
50%	0.6725 ± 0.0043	0.6704 ± 0.0052

so their variance purely reflects randomness in data splitting and training. Figure 1 shows this pattern visually across all fractions.



Summary Table

Labeled %	Manual Acc	PSO Acc	PSO - Manual	Manual Time (s)	PSO Time (s)
5%	0.5048	0.4899	-0.0149	243.9	476.6
10%	0.5580	0.5491	-0.0089	285.2	519.2
15%	0.5828	0.5817	-0.0011	313.3	558.5
20%	0.6092	0.6001	-0.0091	332.1	591.1
30%	0.6419	0.6371	-0.0048	370.6	654.2
50%	0.6725	0.6704	-0.0021	447.0	788.9

Fig. 1: Overview of manual vs. PSO test accuracy across all labeled fractions, with summary statistics.

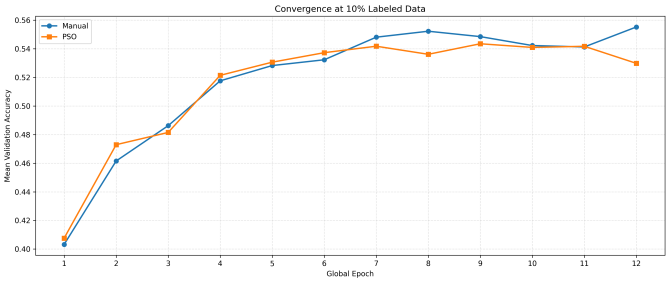
B. PSO Convergence

Figures 2 and 3 plot validation accuracy over global training epochs at 10% and 20% labeled data. Both methods pick up most of their gains in the first SSL round and slow down considerably after that, which is expected — the easiest pseudo-labels get harvested early, and what remains in the unlabeled pool tends to be lower confidence. The manual strategy holds a small but consistent lead across epochs in both regimes, and that lead does not close as training progresses.

C. PSO Best Hyperparameters

Table II lists the configurations PSO converged to at each labeled fraction, averaged over the five seeds. A few patterns are worth pointing out. The threshold τ is consistently in the 0.91–0.95 range rather than at the extremes of our search bounds, which makes intuitive sense — very high thresholds starve the pseudo-label pool, and very low ones flood it with noise. Interestingly, PSO selects a higher τ at 5% than at 10% or 15%, which aligns with the expectation that a weaker model needs to be more selective about which unlabeled samples it trusts.

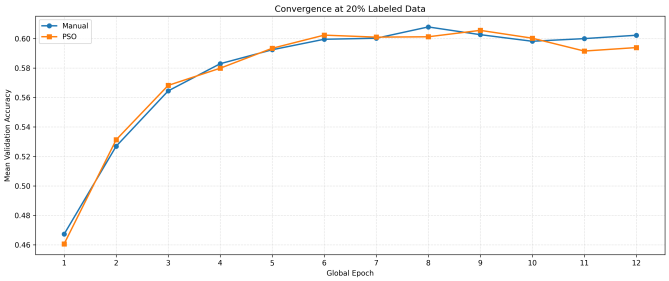
The pseudo-label weight λ_p tends to land around 0.4–0.6 rather than 1.0, which confirms that down-weighting pseudo-labeled samples is a sensible strategy. PSO discovered this



Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.4032	0.4076	+0.0000	+0.0000
2	0.4616	0.4730	+0.0583	+0.0654
3	0.4862	0.4815	+0.0247	+0.0086
4	0.5176	0.5214	+0.0314	+0.0399
5	0.5283	0.5306	+0.0107	+0.0092
6	0.5323	0.5373	+0.0040	+0.0066
7	0.5481	0.5418	+0.0158	+0.0045
8	0.5523	0.5362	+0.0042	-0.0056
9	0.5485	0.5435	-0.0038	+0.0074
10	0.5423	0.5410	-0.0062	-0.0026
11	0.5413	0.5418	-0.0010	+0.0008
12	0.5552	0.5298	+0.0140	-0.0119

Fig. 2: Convergence of validation accuracy at 10% labeled data: manual vs. PSO across global training epochs (mean across 5 seeds).



Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.4673	0.4607	+0.0000	+0.0000
2	0.5269	0.5313	+0.0596	+0.0706
3	0.5645	0.5682	+0.0376	+0.0369
4	0.5829	0.5799	+0.0184	+0.0117
5	0.5924	0.5934	+0.0095	+0.0135
6	0.5995	0.6023	+0.0071	+0.0089
7	0.6002	0.6010	+0.0007	-0.0014
8	0.6079	0.6012	+0.0077	+0.0002
9	0.6026	0.6056	-0.0052	+0.0044
10	0.5982	0.6002	-0.0045	-0.0053
11	0.6000	0.5915	+0.0018	-0.0087
12	0.6022	0.5939	+0.0023	+0.0024

Fig. 3: Convergence of validation accuracy at 20% labeled data: manual vs. PSO across global training epochs (mean across 5 seeds).

without being told — we did not encode any preference for lower weights in the search.

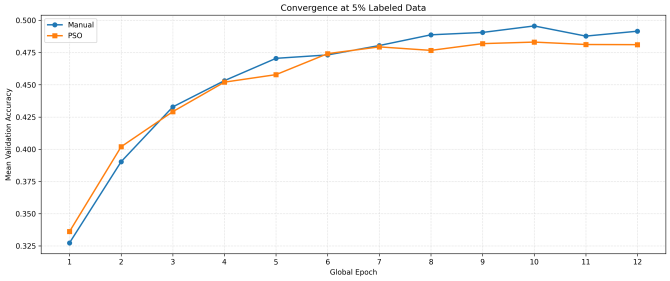
TABLE II: PSO-Optimized Hyperparameters per Labeled Fraction (mean across 5 seeds)

Labeled Fraction	τ	K	λ_p	η
5%	0.9443	1053	0.6645	0.001188
10%	0.9324	762	0.3869	0.001527
15%	0.9195	995	0.4761	0.001167
20%	0.9363	899	0.5347	0.001598
30%	0.9380	481	0.5529	0.000992
50%	0.9381	673	0.5783	0.000985

D. Per-Round SSL Dynamics

Figures 4 and 5 show the two extremes of our labeling regime. At 5% labeled data (Figure 4), both methods are visibly noisier — the seed-to-seed variance is high and neither method improves as smoothly across epochs. This is

not surprising given how little the model has to start from. PSO struggles more here because the fitness landscape is rougher with fewer labels, making it harder for the swarm to consistently find good configurations.

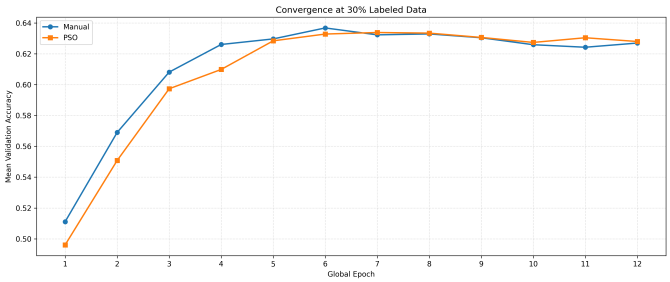


Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.3274	0.3362	+0.0000	+0.0000
2	0.3904	0.4020	+0.0630	+0.0658
3	0.4328	0.4292	+0.0425	+0.0272
4	0.4532	0.4520	+0.0204	+0.0229
5	0.4705	0.4579	+0.0173	+0.0058
6	0.4732	0.4742	+0.0026	+0.0163
7	0.4804	0.4794	+0.0073	+0.0052
8	0.4888	0.4767	+0.0083	-0.0027
9	0.4906	0.4819	+0.0018	+0.0052
10	0.4956	0.4831	+0.0050	+0.0012
11	0.4878	0.4812	-0.0079	-0.0019
12	0.4916	0.4811	+0.0038	-0.0001

Fig. 4: Convergence at 5% labeled data—the most data-scarce regime. Both methods show high variance and slow improvement per epoch.

At 30% labeled data (Figure 5), things look much cleaner. Both methods converge more smoothly and the gap between them shrinks to the point where it is hard to argue either is meaningfully better. This is probably the crossover point where labeled data is abundant enough that the specific pseudo-labeling configuration matters less — the model can afford to be a bit wrong about its pseudo-labels without it hurting much.



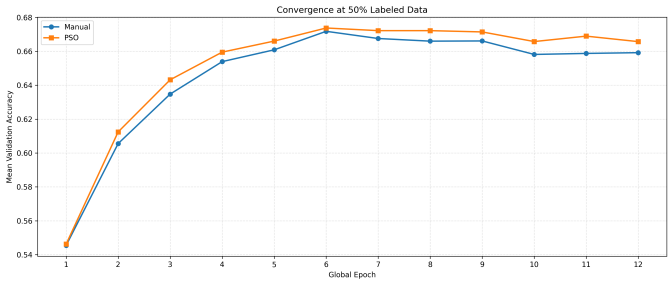
Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.5111	0.4961	+0.0000	+0.0000
2	0.5690	0.5508	+0.0579	+0.0547
3	0.6081	0.5973	+0.0391	+0.0465
4	0.6260	0.6098	+0.0179	+0.0125
5	0.6296	0.6284	+0.0036	+0.0186
6	0.6368	0.6328	+0.0071	+0.0044
7	0.6323	0.6338	-0.0045	+0.0010
8	0.6329	0.6334	+0.0006	-0.0004
9	0.6304	0.6306	-0.0025	-0.0028
10	0.6259	0.6274	-0.0045	-0.0032
11	0.6242	0.6304	-0.0016	+0.0030
12	0.6270	0.6280	+0.0027	-0.0024

Fig. 5: Convergence at 30% labeled data. The gap between manual and PSO narrows as more labeled data is available.

E. Macro F1 Analysis

The macro F1 scores follow the same broad pattern as accuracy, which is reassuring — it means neither method is gaming its accuracy number by over-predicting easy classes. At low labeled fractions the per-class F1 variance is noticeably higher, which just reflects the fact that with very few labeled examples per class, the model's confidence in rarer or visually similar categories (like cats vs. dogs in CIFAR-10) is shaky. As labeled data increases, F1 stabilizes and the two methods converge toward similar per-class performance. Figure 6 illustrates this at the 50% regime, where both methods are essentially indistinguishable.



Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.5455	0.5462	+0.0000	+0.0000
2	0.6056	0.6124	+0.0601	+0.0662
3	0.6348	0.6432	+0.0292	+0.0308
4	0.6539	0.6595	+0.0192	+0.0163
5	0.6609	0.6660	+0.0070	+0.0065
6	0.6718	0.6738	+0.0109	+0.0077
7	0.6676	0.6722	-0.0042	-0.0016
8	0.6660	0.6722	-0.0016	+0.0000
9	0.6661	0.6714	+0.0001	-0.0008
10	0.6582	0.6658	-0.0080	-0.0057
11	0.6588	0.6690	+0.0006	+0.0032
12	0.6592	0.6657	+0.0004	-0.0032

Fig. 6: Convergence at 50% labeled data—the most data-rich regime. Both methods converge quickly and achieve similar final accuracy.

V. Discussion

A. Why Manual Strategies Remain Competitive

Honestly, the fact that manual strategies came out ahead was not entirely surprising to us once we thought about it. When we designed those six configurations, we were not doing so blindly — we knew we were working with CIFAR-10, we knew the model was small, and we had a rough sense from early experiments of what threshold ranges produced reasonable pseudo-label counts. PSO does not get any of that for free. It has to figure out what works by running full SSL evaluations, and with only 30 of those available per labeled fraction, it simply does not have enough budget to thoroughly explore a four-dimensional space.

To put it another way: our manual strategies encoded prior knowledge, and PSO had to earn the same knowledge from scratch. If we had given PSO a swarm of 20 particles over 15 iterations — around 320 evaluations — we suspect it would pull ahead, especially at lower labeled fractions where the landscape is rougher. But that would have taken far more compute than we had available for this project, so the 5-particle, 5-iteration setup was a practical compromise.

B. Computational Cost

The cost difference between the two approaches is significant and worth being upfront about. Each SSL run with $R = 2$ rounds and $E = 3$ epochs per round takes roughly as long as training the supervised baseline for 10 epochs. That means a single PSO run, with its 30 fitness evaluations, costs about $30\times$ what one baseline training run costs. Evaluating all six

manual strategies only needs 6 runs, so PSO ends up being roughly 5× more expensive in terms of total compute.

Whether that cost is worth it really depends on context. In our case, where we are comparing a known dataset with a fixed architecture, it probably is not — the manual strategies already encode enough intuition to do well. But in a new problem setting where no such intuition exists, spending 5× more compute to avoid manual trial-and-error could easily be justified. One direction that would help close this gap is surrogate-assisted PSO, where a cheap surrogate model predicts fitness before committing to a full SSL run, or aggressive early stopping to cut off low-performing configurations before they finish training.

C. Effect of Pseudo-Label Weighting

One thing we were glad to see confirmed by the results is that down-weighting pseudo-labeled samples actually matters. We built $\lambda_p < 1.0$ into the pipeline because it seemed like a sensible guard against noisy labels propagating too strongly into the model. The PSO runs independently arrived at the same conclusion — across all labeled fractions, the swarm converged to weights well below 1.0, typically in the 0.4–0.6 range, without any explicit encouragement to do so.

On the manual strategy side, the configurations that used $\lambda_p = 1.0$ (i.e., treating pseudo-labels as equal in importance to real labels) tended to underperform compared to those that down-weighted them. This is consistent with what has been shown in the noisy-label learning literature [11], where over-relying on uncertain supervision tends to hurt more than it helps once a non-trivial fraction of labels are wrong.

D. Limitations

We want to be clear about where this work has boundaries. The most obvious one is the model size — is lightweight by design, and it is possible that a deeper or wider model would interact differently with pseudo-label noise. Larger models can sometimes absorb noisy supervision better due to having more capacity, or alternatively suffer more from it due to memorization effects; we cannot say which without further experiments.

We also skipped data augmentation entirely. That was a deliberate choice to keep the experimental variables clean, but it means our accuracy numbers are lower than they would be in a more realistic setup, and it is unclear whether augmentation would favor one method over the other. Methods like FixMatch [5] rely heavily on augmentation as part of their core mechanism, so any comparison to those would need to be done carefully.

Finally, everything here is CIFAR-10. It is a clean, balanced, well-studied dataset, and results on it do not automatically generalize to noisier datasets, those with many more classes, or those with significant class imbalance. These are all reasonable directions for follow-up work.

VI. Conclusion

This project set out to answer a fairly concrete question: can PSO find pseudo-labeling hyperparameters that are better than what a person would come up with manually, and if so, at what cost? Based on our experiments across six labeled-data fractions and five random seeds, the honest answer is: not quite, at least not with the compute budget we had. The best manual strategy edged out PSO at every labeled fraction, with the gap being most noticeable at 5% labeled data and shrinking to near-zero by 50%.

What that result actually tells us, though, is less about PSO being a bad algorithm and more about the practical reality of running it on a tight budget. Thirty fitness evaluations spread across a four-dimensional search space is not a lot, and the manual strategies had an unfair advantage in the sense that we designed them with CIFAR-10 and this specific model in mind. Drop PSO into a new problem where that intuition does not exist, and the calculus shifts — spending extra compute to avoid manual trial-and-error starts to look like a reasonable trade.

The other thing the results confirmed is that pseudo-label weighting is not just a nice-to-have. PSO independently discovered that λ_p well below 1.0 works better, which matches our intuition and the broader noisy-label literature. That is a small but satisfying validation of the pipeline design.

Going forward, the most impactful change would probably be surrogate-assisted PSO — using a cheap model to estimate fitness before committing to a full SSL run, which could cut the evaluation budget by an order of magnitude. Adding data augmentation would bring the setup closer to what modern SSL methods actually use, and comparing against FixMatch [5] or FlexMatch would give a clearer picture of where simple pseudo-labeling still holds up and where it falls short.

References

[1] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *Workshop on Challenges in Representation Learning, ICML*, vol. 3, 2013.

[2] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE International Conference on Neural Networks*, vol. 4, pp. 1942–1948, 1995.

[3] X. Zhu, "Semi-supervised learning literature survey," Computer Sciences Technical Report 1530, University of Wisconsin-Madison, 2005.

[4] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-training with noisy student improves ImageNet classification," in *Proc. IEEE/CVF CVPR*, pp. 10687–10698, 2020.

[5] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li, "FixMatch: Simplifying semi-supervised learning with consistency and confidence," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 596–608, 2020.

[6] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems*, vol. 25, pp. 2951–2959, 2012.

[7] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[8] P. R. Lorenzo, J. Nalepa, M. Kawulok, L. S. Ramos, and J. R. Pastor, "Particle swarm optimization for hyper-parameter selection in deep neural networks," in *Proc. Genetic and Evolutionary Computation Conference*, pp. 481–488, 2017.

[9] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, University of Toronto, 2009.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations*, 2015.

[11] N. Natarajan, I. S. Dhillon, P. Ravikumar, and A. Tewari, "Learning with noisy labels," in *Advances in Neural Information Processing Systems*, vol. 26, pp. 1196–1204, 2013.